

DEX WebSocket实时推送系统 - 实现方案

项目背景

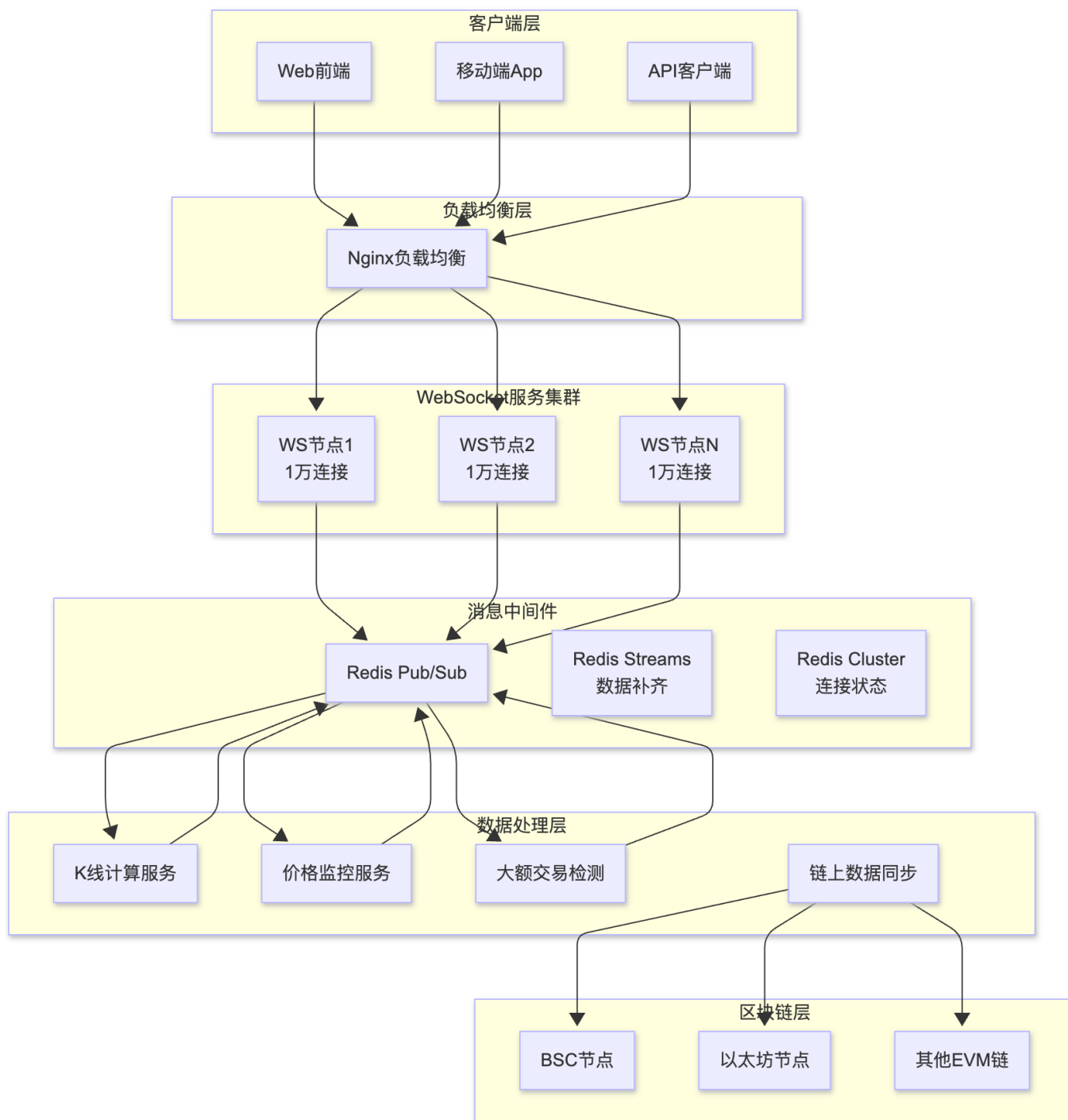
业务场景: AMM交易 + Launchpad代币发行的去中心化交易所

核心需求:

- 实时K线数据推送
 - 价格变动通知
 - 大额交易提醒
 - 支持BSC等多链部署
 - 单机1万+并发连接
-

系统架构设计

整体架构图



核心实现代码

1. WebSocket服务端实现 (Go)

主服务器结构

功能描述:

WebSocket服务端是整个实时推送系统的核心组件，负责管理大量并发连接、处理消息广播、维护连接状态等关键功能。

核心职责:

1. **连接管理:** 维护1万+并发WebSocket连接，包括连接建立、心跳检测、异常断开处理

2. **消息路由**：根据客户端订阅的频道，将相应的消息推送给对应的客户端
3. **数据补齐**：当客户端重连时，根据最后接收的消息序列号，补发缺失的历史消息
4. **负载均衡**：支持多实例部署，通过Redis实现实例间的消息同步
5. **性能监控**：实时统计连接数、消息吞吐量、延迟等关键指标

技术架构：

- 使用Go语言的goroutine实现高并发处理，每个客户端连接对应两个goroutine（读取和写入）
- 采用channel进行goroutine间通信，避免锁竞争
- 使用Redis Pub/Sub实现多实例间的消息广播
- 使用Redis Streams存储消息历史，支持断线重连后的数据补齐

处理流程：

1. 客户端发起WebSocket连接请求
2. 服务端升级HTTP连接为WebSocket连接
3. 创建Client对象，启动读写goroutine
4. 客户端发送订阅请求，服务端记录订阅关系
5. 当有新消息时，服务端根据订阅关系推送给相应客户端
6. 连接断开时，清理相关资源

```
package main

import (
    "context"
    "encoding/json"
    "fmt"
    "log"
    "net/http"
    "strconv"
    "sync"
    "sync/atomic"
    "time"

    "github.com/gin-gonic/gin"
    "github.com/go-redis/redis/v8"
    "github.com/gorilla/websocket"
    "github.com/shopspring/decimal"
)
```

消息类型定义：

系统支持多种业务消息类型，每种类型对应不同的业务场景和数据结构。这种分类设计便于客户端根据业务需求选择性订阅，也便于服务端进行消息路由和处理。

// 业务消息类型

```
const (
    MsgTypeKline      = "kline"           // K线数据更新
    MsgTypePriceAlert = "price_alert"     // 价格提醒通知
    MsgTypeBigTrade   = "big_trade"       // 大额交易提醒
    MsgTypeOrderbook  = "orderbook"       // 订单簿更新
    MsgTypeLaunchpad  = "launchpad"       // Launchpad事件
    MsgTypeHeartbeat  = "heartbeat"       // 心跳消息
)
```

```
MsgTypeBackfill    = "backfill"    // 数据补齐
)
```

****K线数据结构**:**

K线（蜡烛图）是交易所最重要的数据类型之一，包含了特定时间周期内的开盘价、最高价、最低价、收盘价和成交量信息。系统支持多个时间周期（1分钟、5分钟、15分钟、1小时、4小时、1天），实时计算并推送给订阅的客户端。

// K线数据结构

```
type KlineData struct {
    Symbol      string      `json:"symbol"`    // 交易对符号, 如"ETHUSDT"
    Interval    string      `json:"interval"`  // 时间周期, 如"1m", "5m", "1h"
    OpenTime    int64       `json:"openTime"`  // 开盘时间戳
    CloseTime   int64       `json:"closeTime"` // 收盘时间戳
    Open        decimal.Decimal `json:"open"`      // 开盘价
    High        decimal.Decimal `json:"high"`      // 最高价
    Low         decimal.Decimal `json:"low"`       // 最低价
    Close       decimal.Decimal `json:"close"`     // 收盘价
    Volume      decimal.Decimal `json:"volume"`    // 成交量
    Trades      int64        `json:"trades"`    // 成交笔数
}
```

****价格提醒数据结构**:**

价格提醒功能帮助用户监控关注代币的价格变化，当价格达到用户设定的阈值时自动推送通知。系统会实时计算24小时价格变化、涨跌幅度和成交量，为用户提供全面的市场信息。

// 价格提醒数据

```
type PriceAlert struct {
    Symbol          string      `json:"symbol"`    // 交易对符号
    CurrentPrice    decimal.Decimal `json:"currentPrice"` // 当前价格
    Change24h       decimal.Decimal `json:"change24h"`  // 24小时价格变化
    ChangePercent   decimal.Decimal `json:"changePercent"` // 24小时涨跌幅百分比
    Volume24h       decimal.Decimal `json:"volume24h"`  // 24小时成交量
    Timestamp       int64        `json:"timestamp"`  // 时间戳
}
```

****大额交易数据结构**:**

大额交易监控是DEX的重要功能，帮助用户识别市场中的重要交易活动。系统通过监控区块链上的交易，自动识别超过设定阈值的大额交易，并实时推送给用户。这对于分析市场趋势和识别巨鲸活动非常有价值。

// 大额交易数据

```
type BigTrade struct {
    TxHash          string      `json:"txHash"`    // 交易哈希
    Symbol          string      `json:"symbol"`    // 交易对符号
    Type            string      `json:"type"`      // 交易类型: buy/sell
    Amount          decimal.Decimal `json:"amount"`    // 交易数量
    Price           decimal.Decimal `json:"price"`     // 交易价格
    Value           decimal.Decimal `json:"value"`     // USD价值
    Trader          string      `json:"trader"`    // 交易者地址
    Timestamp       int64       `json:"timestamp"` // 交易时间戳
    Chain           string      `json:"chain"`     // 所在区块链
}
```

****Launchpad事件数据结构**:**

Launchpad是代币发射台功能，允许项目方发布新代币并进行初始资金募集。系统需要实时监控Launchpad合约的各种事件，包括代币创建、资金注入、达到目标后自动上线DEX等关键节点，为用户提供及时的投资机会信息。

// Launchpad事件数据

```
type LaunchpadEvent struct {
    TokenAddress string    `json:"tokenAddress"` // 代币合约地址
    TokenName     string    `json:"tokenName"`    // 代币名称
    TokenSymbol   string    `json:"tokenSymbol"`  // 代币符号
    EventType     string    `json:"eventType"`    // 事件类型: created/funded/launched
    Amount        decimal.Decimal `json:"amount"`       // 涉及金额
    Progress      decimal.Decimal `json:"progress"`     // 募集进度百分比(0-100)
    Timestamp     int64      `json:"timestamp"`    // 事件时间戳
    Chain         string     `json:"chain"`        // 所在区块链
}
```

****统一消息结构**:**

所有通过WebSocket传输的消息都使用统一的消息格式，这样设计的好处是便于消息路由、序列化处理和客户端解析。每条消息都包含唯一ID、类型标识、频道信息和序列号，序列号特别重要，用于实现断线重连后的数据补齐功能。

// 统一消息结构

```
type Message struct {
    ID          string    `json:"id"`           // 消息唯一标识符
    Type        string    `json:"type"`         // 消息类型 (kline/price_alert等)
    Channel     string    `json:"channel"`      // 消息频道 (用于订阅路由)
    Data        interface{} `json:"data"`         // 消息具体内容
    Timestamp   int64     `json:"timestamp"`    // 消息时间戳
    Sequence    int64     `json:"sequence"`     // 消息序列号 (用于数据补齐)
}
```

****客户端连接结构**:**

每个WebSocket连接对应一个Client对象，包含了连接的所有状态信息。这个设计支持精细化的连接管理，包括订阅管理、消息发送队列、心跳检测等。使用读写锁确保并发安全，同时记录连接的元信息用于监控和调试。

// 客户端连接

```
type Client struct {
    ID                string    // 客户端唯一标识符
    Conn              *websocket.Conn // WebSocket连接对象
    Send              chan Message   // 消息发送队列 (缓冲256条消息)
    Subscriptions     map[string]bool // 客户端订阅的频道列表
    LastSequence      int64        // 客户端最后接收的消息序列号
    ConnectedAt       time.Time     // 连接建立时间
    LastPingAt        time.Time     // 最后一次心跳时间
    UserAgent         string        // 客户端User-Agent信息
    IPAddress         string        // 客户端IP地址
    mu                sync.RWMutex   // 读写锁, 保护并发访问
}
```

****连接管理器结构**:**

ConnectionManager是整个WebSocket服务的核心组件，负责管理所有客户端连接、处理消息广播、维护统计信息等。采用channel进行goroutine间通信，避免锁竞争，提高并发性能。全局消息序列号确保消息的有序性，统计信息用于监控系统运行状态。

```
// 连接管理器
type ConnectionManager struct {
    clients      map[string]*Client // 所有活跃的客户端连接
    register     chan *Client        // 新连接注册通道
    unregister   chan *Client        // 连接断开注销通道
    broadcast    chan Message        // 消息广播通道（容量1万）
    redis        *redis.Client       // Redis客户端（用于多实例同步）
    sequence     int64              // 全局消息序列号（原子操作）
    mu           sync.RWMutex        // 读写锁保护clients map
}
```

// 统计信息结构

```
stats struct {
    TotalConnections    int64 // 累计连接数
    ActiveConnections  int64 // 当前活跃连接数
    MessagesPerSecond  int64 // 每秒消息数
    BytesPerSecond     int64 // 每秒字节数
    LastStatsUpdate    time.Time // 最后统计更新时间
    mu                 sync.RWMutex // 统计信息锁
}
```

```
}
```

****连接管理器初始化函数**：**

创建ConnectionManager实例并初始化各种channel。channel的容量设计很重要：register和unregister设置为1000，可以处理突发的连接建立和断开；broadcast设置为10000，可以缓冲大量待发送的消息，避免阻塞。同时启动统计协程，定期更新性能指标。

```
func NewConnectionManager(redisClient *redis.Client) *ConnectionManager {
    cm := &ConnectionManager{
        clients:    make(map[string]*Client), // 客户端连接映射
        register:   make(chan *Client, 1000), // 注册通道，缓冲1000个连接请求
        unregister: make(chan *Client, 1000), // 注销通道，缓冲1000个断开请求
        broadcast:  make(chan Message, 10000), // 广播通道，缓冲1万条消息
        redis:      redisClient,              // Redis客户端实例
    }
}
```

// 启动统计协程，每秒更新一次性能指标

```
go cm.updateStats()
```

```
return cm
```

```
}
```

****连接管理器主运行循环**：**

这是ConnectionManager的核心事件循环，使用Go的select语句处理三种类型的事件：新连接注册、连接断开注销、消息广播。这种设计确保了所有操作都在单一goroutine中执行，避免了复杂的锁机制。同时启动Redis订阅协程，接收来自其他实例或数据处理服务的消息。

```
func (cm *ConnectionManager) Run() {
    // 启动Redis订阅协程，监听来自数据处理服务的消息
    go cm.subscribeRedis()

    // 主事件循环，处理连接管理和消息广播
}
```

```

    for {
        select {
            case client := <-cm.register:
                // 处理新客户端连接注册
                cm.registerClient(client)

            case client := <-cm.unregister:
                // 处理客户端连接断开注销
                cm.unregisterClient(client)

            case message := <-cm.broadcast:
                // 处理消息广播到所有订阅的客户端
                cm.broadcastMessage(message)
        }
    }
}

func (cm *ConnectionManager) registerClient(client *Client) {
    cm.mu.Lock()
    cm.clients[client.ID] = client
    cm.mu.Unlock()

    atomic.AddInt64(&cm.stats.ActiveConnections, 1)
    atomic.AddInt64(&cm.stats.TotalConnections, 1)

    log.Printf("Client %s connected from %s. Total: %d",
        client.ID, client.IPAddress, len(cm.clients))

    // 启动客户端处理协程
    go cm.handleClient(client)
    go cm.sendMessages(client)
}

func (cm *ConnectionManager) unregisterClient(client *Client) {
    cm.mu.Lock()
    if _, ok := cm.clients[client.ID]; ok {
        delete(cm.clients, client.ID)
        close(client.Send)
    }
    cm.mu.Unlock()

    atomic.AddInt64(&cm.stats.ActiveConnections, -1)

    log.Printf("Client %s disconnected. Total: %d", client.ID, len(cm.clients))
}

```

****客户端消息处理函数**:**

这个函数负责处理单个客户端的所有消息读取和处理。采用了多重安全机制：设置读取大小限制防止恶意大消息攻击，设置读取超时防止慢速连接占用资源，配置Pong处理器响应心跳检测。使用defer确保连接断开时能正确清理资源。

// 处理客户端消息

```

func (cm *ConnectionManager) handleClient(client *Client) {

```

```

// 使用defer确保连接断开时正确清理资源
defer func() {
    cm.unregister <- client // 发送注销信号
    client.Conn.Close()      // 关闭WebSocket连接
}()

// 设置安全限制, 防止恶意攻击
client.Conn.SetReadLimit(1024) // 限制单条消息最大1KB
client.Conn.SetReadDeadline(time.Now().Add(60 * time.Second)) // 60秒读取超时

// 设置Pong处理器, 响应客户端的心跳检测
client.Conn.SetPongHandler(func(string) error {
    client.Conn.SetReadDeadline(time.Now().Add(60 * time.Second)) // 重置超时
    client.mu.Lock()
    client.LastPingAt = time.Now() // 更新最后心跳时间
    client.mu.Unlock()
    return nil
})

// 消息读取循环
for {
    var request map[string]interface{}
    err := client.Conn.ReadJSON(&request)
    if err != nil {
        // 只记录非正常关闭的错误
        if websocket.IsUnexpectedCloseError(err, websocket.CloseGoingAway,
websocket.CloseAbnormalClosure) {
            log.Printf("WebSocket error for client %s: %v", client.ID, err)
        }
        break
    }

    // 处理客户端请求
    cm.handleClientRequest(client, request)
}

// 处理客户端请求
func (cm *ConnectionManager) handleClientRequest(client *Client, request
map[string]interface{}) {
    msgType, ok := request["type"].(string)
    if !ok {
        return
    }

    switch msgType {
    case "subscribe":
        cm.handleSubscribe(client, request)
    case "unsubscribe":
        cm.handleUnsubscribe(client, request)
    case "backfill_request":
        cm.handleBackfillRequest(client, request)
    }
}

```



```

    case "heartbeat":
        cm.handleHeartbeat(client, request)
    }
}

// 订阅处理
func (cm *ConnectionManager) handleSubscribe(client *Client, request
map[string]interface{}) {
    channels, ok := request["channels"].([]interface{})
    if !ok {
        return
    }

    client.mu.Lock()
    for _, ch := range channels {
        if channel, ok := ch.(string); ok {
            client.Subscriptions[channel] = true
            log.Printf("Client %s subscribed to %s", client.ID, channel)
        }
    }
    client.mu.Unlock()

    // 发送订阅确认
    response := Message{
        ID:      generateMessageID(),
        Type:     "subscription_ack",
        Channel:  "system",
        Data:     map[string]interface{}{"subscribed": channels},
        Timestamp: time.Now().Unix(),
    }

    select {
    case client.Send <- response:
    default:
        log.Printf("Client %s send channel full", client.ID)
    }
}

```

****数据补齐处理函数**:**

当客户端重连时，可能会丢失一些消息。数据补齐功能通过比较客户端最后接收的序列号与服务端的消息历史，找出缺失的消息并重新发送。这个过程在单独的goroutine中异步执行，避免阻塞主处理流程。使用非阻塞发送确保不会因为慢客户端而影响系统性能。

```

// 数据补齐处理
func (cm *ConnectionManager) handleBackfillRequest(client *Client, request
map[string]interface{}) {
    // 解析客户端最后接收的序列号
    lastSequence, ok := request["lastSequence"].(float64)
    if !ok {
        return // 序列号格式错误，忽略请求
    }
}

```

```

    log.Printf("Backfill request from %s, lastSequence: %d", client.ID,
int64(lastSequence))

// 在单独的goroutine中处理数据补齐，避免阻塞主流程
go func() {
    // 从Redis获取缺失的消息
    messages, err := cm.getMessagesSinceSequence(int64(lastSequence))
    if err != nil {
        log.Printf("Error getting backfill messages: %v", err)
        return
    }

    // 如果有缺失的消息，发送给客户端
    if len(messages) > 0 {
        backfillMsg := Message{
            ID:        generateMessageID(),
            Type:       MsgTypeBackfill,
            Channel:   "system",
            Data:       map[string]interface{}{"messages": messages},
            Timestamp: time.Now().Unix(),
        }

        // 非阻塞发送，避免因客户端慢而阻塞
        select {
        case client.Send <- backfillMsg:
            log.Printf("Sent %d backfill messages to %s", len(messages), client.ID)
        default:
            log.Printf("Client %s send channel full during backfill", client.ID)
        }
    }
}()
}

```

****从Redis获取历史消息函数**:**

这个函数实现了数据补齐的核心逻辑，使用Redis Streams来存储和检索消息历史。Redis Streams是一个强大的数据结构，支持按时间顺序存储消息，并且可以根据消息ID进行范围查询。限制每次最多返回1000条消息，避免一次性加载过多数据影响性能。

```

// 从Redis获取历史消息
func (cm *ConnectionManager) getMessagesSinceSequence(lastSequence int64) ([]Message,
error) {
    ctx := context.Background()

    // 使用Redis Streams获取指定序列号之后的消息
    streamName := "dex_messages" // Stream名称
    startID := fmt.Sprintf("%d-0", lastSequence+1) // 起始ID (序列号+1)

    // 执行Redis XREAD命令
    result, err := cm.redis.XRead(ctx, &redis.XReadArgs{
        Streams: []string{streamName, startID}, // Stream名称和起始ID
        Count:   1000,                          // 最多返回1000条消息
        Block:   0,                             // 不阻塞，立即返回
    })

```

```

}).Result()

if err != nil {
    return nil, err
}

// 解析Redis返回的消息
var messages []Message
for _, stream := range result {
    for _, msg := range stream.Messages {
        var message Message
        // 从Redis消息中提取JSON数据
        if data, ok := msg.Values["data"].(string); ok {
            if err := json.Unmarshal([]byte(data), &message); err == nil {
                messages = append(messages, message)
            }
        }
    }
}

return messages, nil
}

// 心跳处理
func (cm *ConnectionManager) handleHeartbeat(client *Client, request
map[string]interface{}) {
    response := Message{
        ID:      generateMessageID(),
        Type:     MsgTypeHeartbeat,
        Channel:  "system",
        Data:     map[string]interface{}{"timestamp": time.Now().Unix()},
        Timestamp: time.Now().Unix(),
    }

    select {
    case client.Send <- response:
    default:
        // 心跳发送失败, 可能需要断开连接
    }
}

// 发送消息到客户端
func (cm *ConnectionManager) sendMessages(client *Client) {
    ticker := time.NewTicker(54 * time.Second) // Ping间隔
    defer func() {
        ticker.Stop()
        client.Conn.Close()
    }()

    for {
        select {
        case message, ok := <-client.Send:

```

```

    client.Conn.SetWriteDeadline(time.Now().Add(10 * time.Second))
    if !ok {
        client.Conn.WriteMessage(websocket.CloseMessage, []byte{})
        return
    }

    // 更新客户端最后序列号
    if message.Sequence > 0 {
        client.mu.Lock()
        client.LastSequence = message.Sequence
        client.mu.Unlock()
    }

    if err := client.Conn.WriteJSON(message); err != nil {
        log.Printf("Write error for client %s: %v", client.ID, err)
        return
    }

    // 更新统计
    atomic.AddInt64(&cm.stats.MessagesPerSecond, 1)

case <-ticker.C:
    client.Conn.SetWriteDeadline(time.Now().Add(10 * time.Second))
    if err := client.Conn.WriteMessage(websocket.PingMessage, nil); err != nil {
        log.Printf("Ping error for client %s: %v", client.ID, err)
        return
    }
}
}
}

// 广播消息
func (cm *ConnectionManager) broadcastMessage(message Message) {
    // 设置序列号
    message.Sequence = atomic.AddInt64(&cm.sequence, 1)

    // 保存到Redis Stream用于数据补齐
    cm.saveMessageToRedis(message)

    cm.mu.RLock()
    defer cm.mu.RUnlock()

    for _, client := range cm.clients {
        if cm.isClientSubscribed(client, message.Channel) {
            select {
            case client.Send <- message:
            default:
                // 客户端发送队列满，断开连接
                go func(c *Client) {
                    cm.unregister <- c
                }(client)
            }
        }
    }
}

```

```

    }
}

// 保存消息到Redis
func (cm *ConnectionManager) saveMessageToRedis(message Message) {
    ctx := context.Background()

    data, err := json.Marshal(message)
    if err != nil {
        log.Printf("Error marshaling message: %v", err)
        return
    }

    // 保存到Redis Stream
    streamName := "dex_messages"
    _, err = cm.redis.XAdd(ctx, &redis.XAddArgs{
        Stream: streamName,
        ID:      fmt.Sprintf("%d-*", message.Sequence),
        Values: map[string]interface{}{
            "data": string(data),
        },
    }).Result()

    if err != nil {
        log.Printf("Error saving message to Redis: %v", err)
    }

    // 限制Stream长度, 保留最近24小时的数据
    cm.redis.XTrimMaxLen(ctx, streamName, 1000000) // 保留100万条消息
}

// Redis订阅
func (cm *ConnectionManager) subscribeRedis() {
    ctx := context.Background()
    pubsub := cm.redis.Subscribe(ctx,
        "kline_updates",
        "price_alerts",
        "big_trades",
        "launchpad_events",
    )
    defer pubsub.Close()

    for {
        msg, err := pubsub.ReceiveMessage(ctx)
        if err != nil {
            log.Printf("Redis subscription error: %v", err)
            time.Sleep(time.Second)
            continue
        }

        var message Message

```

```

        if err := json.Unmarshal([]byte(msg.Payload), &message); err != nil {
            log.Printf("Error unmarshaling Redis message: %v", err)
            continue
        }

        select {
        case cm.broadcast <- message:
        default:
            log.Printf("Broadcast channel full, dropping message")
        }
    }
}

func (cm *ConnectionManager) isClientSubscribed(client *Client, channel string) bool {
    client.mu.RLock()
    defer client.mu.RUnlock()
    return client.Subscriptions[channel]
}

// 统计更新
func (cm *ConnectionManager) updateStats() {
    ticker := time.NewTicker(time.Second)
    defer ticker.Stop()

    var lastMessages int64

    for range ticker.C {
        cm.stats.mu.Lock()

        currentMessages := atomic.LoadInt64(&cm.stats.MessagesPerSecond)
        cm.stats.MessagesPerSecond = currentMessages - lastMessages
        lastMessages = currentMessages

        cm.stats.LastStatsUpdate = time.Now()
        cm.stats.mu.Unlock()
    }
}

// WebSocket升级器
var upgrader = websocket.Upgrader{
    ReadBufferSize: 4096,
    WriteBufferSize: 4096,
    CheckOrigin: func(r *http.Request) bool {
        // 生产环境需要严格的Origin检查
        return true
    },
}

// HTTP处理器
func handleWebSocket(cm *ConnectionManager) gin.HandlerFunc {
    return func(c *gin.Context) {
        conn, err := upgrader.Upgrade(c.Writer, c.Request, nil)
    }
}

```

```

    if err != nil {
        log.Printf("WebSocket upgrade failed: %v", err)
        return
    }

    client := &Client{
        ID:          generateClientID(),
        Conn:        conn,
        Send:        make(chan Message, 256),
        Subscriptions: make(map[string]bool),
        ConnectedAt: time.Now(),
        LastPingAt:  time.Now(),
        UserAgent:   c.Request.UserAgent(),
        IPAddress:   c.ClientIP(),
    }

    cm.register <- client
}

func generateClientID() string {
    return fmt.Sprintf("client_%d_%d", time.Now().UnixNano(), rand.Intn(10000))
}

func generateMessageID() string {
    return fmt.Sprintf("msg_%d_%d", time.Now().UnixNano(), rand.Intn(1000))
}

```

2. 数据处理服务

K线数据处理服务

功能描述：

K线数据处理服务是DEX系统的重要组成部分，负责实时计算和更新各种时间周期的K线数据。当有新的交易发生时，系统会自动更新相应的K线数据并通过Redis发布给WebSocket服务进行广播。

核心功能：

1. **多周期支持：**同时维护1分钟、5分钟、15分钟、1小时、4小时、1天等多个时间周期的K线
2. **实时计算：**每笔交易都会触发相关K线的实时更新，包括开盘价、最高价、最低价、收盘价、成交量
3. **数据持久化：**使用Redis Hash结构存储K线数据，支持快速查询和更新
4. **消息发布：**K线更新后立即通过Redis Pub/Sub发布消息，通知所有WebSocket实例
5. **数据过期：**自动清理过期的K线数据，避免Redis内存无限增长

处理流程：

1. 接收交易数据（价格、数量、时间戳等）
2. 根据交易时间计算各个时间周期的K线时间戳
3. 从Redis获取现有K线数据或创建新的K线
4. 更新K线的OHLCV数据（开盘价、最高价、最低价、收盘价、成交量）
5. 将更新后的K线数据保存到Redis
6. 发布K线更新消息到Redis Pub/Sub

```

package main

import (
    "context"
    "encoding/json"
    "log"
    "time"

    "github.com/go-redis/redis/v8"
    "github.com/shopspring/decimal"
)

type KlineProcessor struct {
    redis    *redis.Client
    intervals []string // 1m, 5m, 15m, 1h, 4h, 1d
}

func NewKlineProcessor(redisClient *redis.Client) *KlineProcessor {
    return &KlineProcessor{
        redis:    redisClient,
        intervals: []string{"1m", "5m", "15m", "1h", "4h", "1d"},
    }
}

func (kp *KlineProcessor) ProcessTrade(trade TradeData) {
    ctx := context.Background()

    // 为每个时间间隔更新K线
    for _, interval := range kp.intervals {
        klineKey := fmt.Sprintf("kline:%s:%s", trade.Symbol, interval)
        timestamp := kp.getKlineTimestamp(trade.Timestamp, interval)

        // 使用Redis Hash存储K线数据
        pipe := kp.redis.Pipeline()

        // 更新或创建K线
        klineData := kp.updateKlineData(ctx, klineKey, timestamp, trade)

        // 发布K线更新
        message := Message{
            ID:          generateMessageID(),
            Type:        MsgTypeKline,
            Channel:     fmt.Sprintf("kline_%s_%s", trade.Symbol, interval),
            Data:        klineData,
            Timestamp:   time.Now().Unix(),
        }

        msgData, _ := json.Marshal(message)
        pipe.Publish(ctx, "kline_updates", string(msgData))

        pipe.Exec(ctx)
    }
}

```



```

}

func (kp *KlineProcessor) updateKlineData(ctx context.Context, key string, timestamp
int64, trade TradeData) *KlineData {
    timestampStr := fmt.Sprintf("%d", timestamp)

    // 获取现有K线数据
    existing := kp.redis.HGetAll(ctx, key+": "+timestampStr).Val()

    var kline *KlineData
    if len(existing) == 0 {
        // 创建新K线
        kline = &KlineData{
            Symbol:    trade.Symbol,
            OpenTime:  timestamp,
            CloseTime: timestamp + kp.getIntervalDuration("1m") - 1,
            Open:      trade.Price,
            High:      trade.Price,
            Low:        trade.Price,
            Close:      trade.Price,
            Volume:     trade.Amount,
            Trades:    1,
        }
    } else {
        // 更新现有K线
        kline = &KlineData{
            Symbol:    trade.Symbol,
            OpenTime:  timestamp,
        }

        // 解析现有数据
        if open, err := decimal.NewFromString(existing["open"]); err == nil {
            kline.Open = open
        }
        if high, err := decimal.NewFromString(existing["high"]); err == nil {
            kline.High = high
        }
        if low, err := decimal.NewFromString(existing["low"]); err == nil {
            kline.Low = low
        }
        if volume, err := decimal.NewFromString(existing["volume"]); err == nil {
            kline.Volume = volume
        }

        // 更新数据
        kline.Close = trade.Price
        if trade.Price.GreaterThan(kline.High) {
            kline.High = trade.Price
        }
        if trade.Price.LessThan(kline.Low) {
            kline.Low = trade.Price
        }
    }
}

```

```

        kline.Volume = kline.Volume.Add(trade.Amount)
        kline.Trades++
    }

    // 保存更新后的k线数据
    kp.redis.HMSet(ctx, key+": "+timestampStr, map[string]interface{}{
        "open":    kline.Open.String(),
        "high":    kline.High.String(),
        "low":     kline.Low.String(),
        "close":   kline.Close.String(),
        "volume":  kline.Volume.String(),
        "trades":  kline.Trades,
    })

    // 设置过期时间
    kp.redis.Expire(ctx, key+": "+timestampStr, 30*24*time.Hour) // 30天

    return kline
}

func (kp *KlineProcessor) getKlineTimestamp(timestamp int64, interval string) int64 {
    duration := kp.getIntervalDuration(interval)
    return (timestamp / duration) * duration
}

func (kp *KlineProcessor) getIntervalDuration(interval string) int64 {
    switch interval {
    case "1m":
        return 60
    case "5m":
        return 300
    case "15m":
        return 900
    case "1h":
        return 3600
    case "4h":
        return 14400
    case "1d":
        return 86400
    default:
        return 60
    }
}

```

大额交易监控服务

功能描述：

大额交易监控服务通过实时监控区块链上的交易活动，自动识别和报告超过预设阈值的大额交易。这对于DEX用户了解市场动态、识别巨鲸活动、分析价格影响因素非常重要。

核心功能：

1. **多链监控**: 同时监控以太坊、BSC等多个区块链网络的交易活动
2. **实时检测**: 通过WebSocket连接到区块链节点, 实时接收新区块和交易数据
3. **智能过滤**: 只关注与DEX相关的交易, 过滤掉无关的转账和合约调用
4. **阈值判断**: 根据不同代币设置不同的大额交易阈值, 灵活适应各种资产
5. **即时通知**: 检测到大额交易后立即通过Redis发布消息, 实时通知用户

监控流程:

1. 建立与区块链节点的WebSocket连接
2. 订阅新区块头事件, 获取最新区块信息
3. 遍历区块中的所有交易, 检查交易目标地址
4. 识别DEX相关交易 (通过合约地址和方法签名)
5. 解析交易数据, 提取交易金额、代币类型等信息
6. 与预设阈值比较, 判断是否为大额交易
7. 构造大额交易消息并发布到Redis

技术特点:

- 使用go-ethereum库与区块链节点交互
- 支持多个DEX协议 (Uniswap、PancakeSwap等)
- 实时计算交易的USD价值
- 记录交易者地址和交易哈希便于追踪

```
package main

import (
    "context"
    "encoding/json"
    "log"
    "math/big"

    "github.com/ethereum/go-ethereum"
    "github.com/ethereum/go-ethereum/common"
    "github.com/ethereum/go-ethereum/core/types"
    "github.com/ethereum/go-ethereum/ethclient"
    "github.com/go-redis/redis/v8"
    "github.com/shopspring/decimal"
)

type BigTradeMonitor struct {
    redis      *redis.Client
    ethClient  *ethclient.Client
    bscClient  *ethclient.Client
    thresholds map[string]decimal.Decimal // 每个代币的大额交易阈值
}

func NewBigTradeMonitor(redisClient *redis.Client) *BigTradeMonitor {
    ethClient, _ := ethclient.Dial("wss://mainnet.infura.io/ws/v3/YOUR_KEY")
    bscClient, _ := ethclient.Dial("wss://bsc-ws-node.nariox.org:443")

    return &BigTradeMonitor{
```

```

        redis:    redisClient,
        ethClient: ethClient,
        bscClient: bscClient,
        thresholds: map[string]decimal.Decimal{
            "ETH": decimal.NewFromInt(10),    // 10 ETH
            "BNB": decimal.NewFromInt(100),  // 100 BNB
            "USDT": decimal.NewFromInt(50000), // 50,000 USDT
        },
    },
}

func (btm *BigTradeMonitor) Start() {
    // 监控以太坊
    go btm.monitorChain(btm.ethClient, "ethereum")

    // 监控BSC
    go btm.monitorChain(btm.bscClient, "bsc")
}

func (btm *BigTradeMonitor) monitorChain(client *ethclient.Client, chainName string) {
    ctx := context.Background()

    // 订阅新区块
    headers := make(chan *types.Header)
    sub, err := client.SubscribeNewHead(ctx, headers)
    if err != nil {
        log.Printf("Failed to subscribe to %s headers: %v", chainName, err)
        return
    }
    defer sub.Unsubscribe()

    for {
        select {
        case err := <-sub.Err():
            log.Printf("%s subscription error: %v", chainName, err)
            return

        case header := <-headers:
            btm.processBlock(ctx, client, header, chainName)
        }
    }
}

func (btm *BigTradeMonitor) processBlock(ctx context.Context, client *ethclient.Client,
header *types.Header, chain string) {
    block, err := client.BlockByHash(ctx, header.Hash())
    if err != nil {
        log.Printf("Failed to get block: %v", err)
        return
    }

    for _, tx := range block.Transactions() {

```

```

        btm.analyzeTx(ctx, tx, chain)
    }
}

func (btm *BigTradeMonitor) analyzeTx(ctx context.Context, tx *types.Transaction, chain
string) {
    // 检查是否是DEX交易（通过合约地址和方法签名）
    if !btm.isDEXTransaction(tx) {
        return
    }

    // 解析交易数据
    trade := btm.parseTradeData(tx, chain)
    if trade == nil {
        return
    }

    // 检查是否为大额交易
    if btm.isBigTrade(trade) {
        btm.publishBigTrade(ctx, trade)
    }
}

func (btm *BigTradeMonitor) isDEXTransaction(tx *types.Transaction) bool {
    // 检查常见DEX合约地址
    dexContracts := map[common.Address]bool{
        common.HexToAddress("0x7a250d5630B4cF539739dF2C5dAcb4c659F2488D"): true, //
Uniswap V2
        common.HexToAddress("0xE592427A0AEce92De3Edee1F18E0157C05861564"): true, //
Uniswap V3
        common.HexToAddress("0x10ED43C718714eb63d5aA57B78B54704E256024E"): true, //
PancakeSwap
    }

    return tx.To() != nil && dexContracts[*tx.To()]
}

func (btm *BigTradeMonitor) parseTradeData(tx *types.Transaction, chain string) *BigTrade
{
    // 这里需要根据具体的DEX合约ABI解析交易数据
    // 简化示例
    return &BigTrade{
        TxHash:    tx.Hash().Hex(),
        Symbol:    "ETH/USDT", // 需要从交易数据中解析
        Type:      "buy",      // 需要从交易数据中解析
        Amount:    decimal.NewFromBigInt(tx.Value(), -18),
        Price:     decimal.NewFromInt(3000), // 需要从交易数据中解析
        Value:     decimal.NewFromInt(30000), // 计算USD价值
        Trader:    tx.From().Hex(),
        Timestamp: time.Now().Unix(),
        Chain:     chain,
    }
}

```

```

}

func (btm *BigTradeMonitor) isBigTrade(trade *BigTrade) bool {
    threshold, exists := btm.thresholds[trade.Symbol]
    if !exists {
        threshold = decimal.NewFromInt(10000) // 默认$10,000
    }

    return trade.Value.GreaterThanOrEqual(threshold)
}

func (btm *BigTradeMonitor) publishBigTrade(ctx context.Context, trade *BigTrade) {
    message := Message{
        ID:          generateMessageID(),
        Type:         MsgTypeBigTrade,
        Channel:      fmt.Sprintf("big_trades_%s", trade.Symbol),
        Data:         trade,
        Timestamp:    time.Now().Unix(),
    }

    msgData, _ := json.Marshal(message)
    btm.redis.Publish(ctx, "big_trades", string(msgData))

    log.Printf("Big trade detected: %s %s %s for %s",
        trade.Type, trade.Amount.String(), trade.Symbol, trade.Value.String())
}

```

3. 客户端自动重连实现

JavaScript客户端

功能描述：

DEXWebSocket是一个功能完整的JavaScript客户端库，专门为DEX实时数据推送设计。它实现了智能重连、数据补齐、消息分类处理等高级功能，确保用户能够稳定可靠地接收实时市场数据。

核心特性：

1. **智能重连：**采用指数退避算法，在网络断开时自动重连，避免频繁重连造成服务器压力
2. **数据补齐：**重连后自动请求缺失的历史数据，确保数据的完整性和连续性
3. **消息分类：**根据消息类型自动分发到不同的处理函数，简化业务逻辑
4. **订阅管理：**支持动态订阅和取消订阅，重连后自动恢复订阅状态
5. **心跳检测：**定期发送心跳消息，监控连接状态和网络延迟
6. **浏览器通知：**支持浏览器原生通知，及时提醒用户重要事件

重连策略：

- 初始重连间隔：1秒
- 最大重连间隔：30秒
- 退避系数：1.5（每次失败后间隔时间乘以1.5）
- 支持无限重连或设置最大重连次数

数据补齐机制：

- 记录最后接收的消息序列号
- 重连时发送补齐请求
- 服务端返回缺失的历史消息
- 按序列号排序后重新处理

```
class DEXWebSocket {
  constructor(url, options = {}) {
    this.url = url;
    this.options = {
      maxReconnectAttempts: options.maxReconnectAttempts || -1,
      reconnectInterval: options.reconnectInterval || 1000,
      maxReconnectInterval: options.maxReconnectInterval || 30000,
      reconnectDecay: options.reconnectDecay || 1.5,
      heartbeatInterval: options.heartbeatInterval || 30000,
      ...options
    };

    this.reconnectAttempts = 0;
    this.forcedClose = false;
    this.eventListeners = {};
    this.subscriptions = new Set();
    this.lastSequence = 0;
    this.messageBuffer = [];

    this.connect();
  }

  connect() {
    console.log(`Connecting to DEX WebSocket: ${this.url}`);

    this.ws = new WebSocket(this.url);

    this.ws.onopen = (event) => {
      console.log('DEX WebSocket connected');
      this.reconnectAttempts = 0;

      // 重新订阅频道
      this.resubscribe();

      // 请求数据补齐
      this.requestBackfill();

      // 启动心跳
      this.startHeartbeat();

      this.fireEvent('open', event);
    };

    this.ws.onclose = (event) => {
      console.log('DEX WebSocket closed', event.code, event.reason);
      this.stopHeartbeat();
    };
  }
}
```

```

        if (!this.forcedClose) {
            this.fireEvent('close', event);
            this.scheduleReconnect();
        }
    };

    this.ws.onerror = (event) => {
        console.error('DEX WebSocket error:', event);
        this.fireEvent('error', event);
    };

    this.ws.onmessage = (event) => {
        this.handleMessage(event);
    };
}

handleMessage(event) {
    try {
        const message = JSON.parse(event.data);

        // 更新最后序列号
        if (message.sequence) {
            this.lastSequence = Math.max(this.lastSequence, message.sequence);
        }

        // 处理不同类型的消息
        switch (message.type) {
            case 'kline':
                this.handleKlineUpdate(message);
                break;
            case 'price_alert':
                this.handlePriceAlert(message);
                break;
            case 'big_trade':
                this.handleBigTrade(message);
                break;
            case 'launchpad':
                this.handleLaunchpadEvent(message);
                break;
            case 'backfill':
                this.handleBackfillData(message);
                break;
            case 'heartbeat':
                this.handleHeartbeat(message);
                break;
            default:
                this.fireEvent('message', { data: event.data });
        }
    } catch (error) {
        console.error('Error parsing message:', error);
    }
}

```



```

}

handleKlineUpdate(message) {
  console.log('K线更新:', message.data);
  this.fireEvent('kline', message.data);
}

handlePriceAlert(message) {
  console.log('价格提醒:', message.data);
  this.fireEvent('priceAlert', message.data);

  // 可以在这里添加浏览器通知
  if (Notification.permission === 'granted') {
    new Notification(`价格提醒: ${message.data.symbol}`, {
      body: `当前价格: ${message.data.currentPrice}`,
      icon: '/favicon.ico'
    });
  }
}

handleBigTrade(message) {
  console.log('大额交易:', message.data);
  this.fireEvent('bigTrade', message.data);

  // 大额交易特殊提醒
  if (Notification.permission === 'granted') {
    new Notification(`大额交易提醒`, {
      body: `${message.data.type.toUpperCase()} ${message.data.amount}
${message.data.symbol} 价值 ${message.data.value}`,
      icon: '/favicon.ico'
    });
  }
}

handleLaunchpadEvent(message) {
  console.log('Launchpad事件:', message.data);
  this.fireEvent('launchpad', message.data);
}

handleBackfillData(message) {
  console.log(`收到 ${message.data.messages.length} 条补齐消息`);

  // 按序列号排序并处理
  message.data.messages
    .sort((a, b) => a.sequence - b.sequence)
    .forEach(msg => {
      this.handleMessage({ data: JSON.stringify(msg) });
    });
}

handleHeartbeat(message) {
  const latency = Date.now() - message.data.timestamp * 1000;

```

```

        console.log(`心跳延迟: ${latency}ms`);
        this.fireEvent('heartbeat', { latency });
    }

    // 订阅频道
    subscribe(channels) {
        if (!Array.isArray(channels)) {
            channels = [channels];
        }

        channels.forEach(channel => this.subscriptions.add(channel));

        if (this.ws && this.ws.readyState === WebSocket.OPEN) {
            this.send({
                type: 'subscribe',
                channels: channels
            });
        }
    }

    // 取消订阅
    unsubscribe(channels) {
        if (!Array.isArray(channels)) {
            channels = [channels];
        }

        channels.forEach(channel => this.subscriptions.delete(channel));

        if (this.ws && this.ws.readyState === WebSocket.OPEN) {
            this.send({
                type: 'unsubscribe',
                channels: channels
            });
        }
    }

    // 重新订阅所有频道
    resubscribe() {
        if (this.subscriptions.size > 0) {
            this.send({
                type: 'subscribe',
                channels: Array.from(this.subscriptions)
            });
        }
    }

    // 请求数据补齐
    requestBackfill() {
        if (this.lastSequence > 0) {
            this.send({
                type: 'backfill_request',
                lastSequence: this.lastSequence
            });
        }
    }

```

```

    });
  }
}

// 启动心跳
startHeartbeat() {
  this.heartbeatTimer = setInterval(() => {
    if (this.ws && this.ws.readyState === WebSocket.OPEN) {
      this.send({
        type: 'heartbeat',
        timestamp: Math.floor(Date.now() / 1000)
      });
    }
  }, this.options.heartbeatInterval);
}

stopHeartbeat() {
  if (this.heartbeatTimer) {
    clearInterval(this.heartbeatTimer);
    this.heartbeatTimer = null;
  }
}

scheduleReconnect() {
  if (this.options.maxReconnectAttempts > -1 &&
    this.reconnectAttempts >= this.options.maxReconnectAttempts) {
    console.log('达到最大重连次数');
    return;
  }

  this.reconnectAttempts++;

  const timeout = Math.min(
    this.options.reconnectInterval * Math.pow(this.options.reconnectDecay,
this.reconnectAttempts - 1),
    this.options.maxReconnectInterval
  );

  console.log(`${timeout}ms 后重连 (第 ${this.reconnectAttempts} 次尝试)`);

  setTimeout(() => {
    if (!this.forcedClose) {
      this.connect();
    }
  }, timeout);
}

send(data) {
  if (this.ws && this.ws.readyState === WebSocket.OPEN) {
    this.ws.send(JSON.stringify(data));
  } else {
    console.log('WebSocket未连接, 消息已缓存');
  }
}

```

```

        this.messageBuffer.push(data);
    }
}

close() {
    this.forcedClose = true;
    this.stopHeartbeat();
    if (this.ws) {
        this.ws.close();
    }
}

// 事件处理
addEventListener(type, listener) {
    if (!this.eventListeners[type]) {
        this.eventListeners[type] = [];
    }
    this.eventListeners[type].push(listener);
}

removeEventListener(type, listener) {
    if (this.eventListeners[type]) {
        const index = this.eventListeners[type].indexOf(listener);
        if (index > -1) {
            this.eventListeners[type].splice(index, 1);
        }
    }
}

fireEvent(type, data) {
    if (this.eventListeners[type]) {
        this.eventListeners[type].forEach(listener => {
            try {
                listener(data);
            } catch (error) {
                console.error('事件监听器错误:', error);
            }
        });
    }
}

// 使用示例
const dexWS = new DEXWebSocket('wss://api.yourDEX.com/ws');

// 监听连接事件
dexWS.addEventListener('open', () => {
    console.log('已连接到DEX');
});

// 订阅K线数据
dexWS.subscribe([
    'kline_ETHUSDT_1m',

```

```

        'kline_ETHUSD_5m',
        'kline_BNBUSD_1m'
    ]);

    // 订阅价格提醒
    dexWS.subscribe(['price_alerts']);

    // 订阅大额交易
    dexWS.subscribe(['big_trades_ETHUSD', 'big_trades_BNBUSD']);

    // 订阅Launchpad事件
    dexWS.subscribe(['launchpad_events']);
});

// 监听K线更新
dexWS.addEventListener('kline', (klineData) => {
    console.log('K线更新:', klineData);
    // 更新图表
    updateChart(klineData);
});

// 监听价格提醒
dexWS.addEventListener('priceAlert', (alert) => {
    console.log('价格提醒:', alert);
    // 更新价格显示
    updatePriceDisplay(alert);
});

// 监听大额交易
dexWS.addEventListener('bigTrade', (trade) => {
    console.log('大额交易:', trade);
    // 显示大额交易通知
    showBigTradeNotification(trade);
});

// 监听Launchpad事件
dexWS.addEventListener('launchpad', (event) => {
    console.log('Launchpad事件:', event);
    // 更新Launchpad状态
    updateLaunchpadStatus(event);
});

// 请求浏览器通知权限
if (Notification.permission === 'default') {
    Notification.requestPermission();
}

```

部署与优化

1. 系统配置优化

Linux系统优化

优化目标：

为了支持1万+并发WebSocket连接，需要对Linux系统进行深度优化。默认的系统配置通常只能支持几百到几千个并发连接，通过调整内核参数、文件描述符限制、网络缓冲区等设置，可以显著提升系统的并发处理能力。

优化内容：

1. **文件描述符限制**：每个WebSocket连接都需要一个文件描述符，默认限制通常为1024，需要提升到65536
2. **网络连接队列**：增加TCP连接队列长度，避免连接请求被丢弃
3. **网络缓冲区**：优化TCP读写缓冲区大小，提高网络I/O性能
4. **连接超时设置**：调整TCP连接的超时参数，快速释放无效连接

执行方式：

将以下脚本保存为 `optimize_system.sh`，使用 `sudo bash optimize_system.sh` 执行，然后重启系统使配置生效。

```
#!/bin/bash
# 系统优化脚本 - 支持1万+并发WebSocket连接

# 增加文件描述符限制
echo "* soft nofile 65536" >> /etc/security/limits.conf
echo "* hard nofile 65536" >> /etc/security/limits.conf

# 网络优化
echo "net.core.somaxconn = 65536" >> /etc/sysctl.conf
echo "net.ipv4.tcp_max_syn_backlog = 65536" >> /etc/sysctl.conf
echo "net.core.netdev_max_backlog = 5000" >> /etc/sysctl.conf
echo "net.ipv4.tcp_fin_timeout = 30" >> /etc/sysctl.conf
echo "net.ipv4.tcp_keepalive_time = 1200" >> /etc/sysctl.conf
echo "net.ipv4.tcp_rmem = 4096 65536 16777216" >> /etc/sysctl.conf
echo "net.ipv4.tcp_wmem = 4096 65536 16777216" >> /etc/sysctl.conf

sysctl -p
```

Docker部署配置

部署架构说明：

使用Docker容器化部署可以确保环境一致性，简化部署流程，并支持水平扩展。整个系统采用微服务架构，包括WebSocket服务、Redis集群、Nginx负载均衡器等组件。

容器配置要点：

1. **WebSocket服务**：部署3个实例，每个实例支持1万连接，总计3万并发
2. **资源限制**：为每个容器设置合理的CPU和内存限制，避免资源争抢
3. **环境变量**：通过环境变量配置Redis连接、区块链节点URL等参数
4. **健康检查**：配置容器健康检查，确保服务正常运行
5. **数据持久化**：Redis数据通过Volume持久化，避免重启丢失数据

扩展能力：

- 可以通过增加 `replicas` 数量来水平扩展WebSocket服务

- 支持滚动更新，零停机部署新版本
- 通过Docker Swarm或Kubernetes进行集群管理

```
# docker-compose.yml - 生产级部署配置
```

```
version: '3.8'
```

```
services:
```

```
  dex-websocket:
```

```
    build: .
```

```
    ports:
```

```
      - "8080:8080"
```

```
    environment:
```

```
      - REDIS_URL=redis://redis:6379
```

```
      - ETH_WS_URL=wss://mainnet.infura.io/ws/v3/YOUR_KEY
```

```
      - BSC_WS_URL=wss://bsc-ws-node.nariox.org:443
```

```
      - MAX_CONNECTIONS=10000
```

```
    deploy:
```

```
      replicas: 3
```

```
      resources:
```

```
        limits:
```

```
          memory: 2G
```

```
          cpus: '1.0'
```

```
        reservations:
```

```
          memory: 1G
```

```
          cpus: '0.5'
```

```
    depends_on:
```

```
      - redis
```

```
  nginx:
```

```
    image: nginx:alpine
```

```
    ports:
```

```
      - "80:80"
```

```
      - "443:443"
```

```
    volumes:
```

```
      - ./nginx.conf:/etc/nginx/nginx.conf
```

```
      - ./ssl:/etc/nginx/ssl
```

```
    depends_on:
```

```
      - dex-websocket
```

```
  redis:
```

```
    image: redis:7-alpine
```

```
    ports:
```

```
      - "6379:6379"
```

```
    command: redis-server --maxmemory 2gb --maxmemory-policy allkeys-lru
```

```
    volumes:
```

```
      - redis_data:/data
```

```
  redis-sentinel:
```

```
    image: redis:7-alpine
```

```
    command: redis-sentinel /etc/redis/sentinel.conf
```

```
    volumes:
```

```
      - ./sentinel.conf:/etc/redis/sentinel.conf
```

```
depends_on:
  - redis

volumes:
  redis_data:
```

2. Nginx负载均衡配置

负载均衡策略说明：

Nginx作为反向代理和负载均衡器，是整个系统的入口点。对于WebSocket连接，需要特殊的配置来确保连接的稳定性和会话保持。

关键配置要点：

1. **会话粘性**：使用一致性哈希确保同一客户端的重连请求路由到同一个后端服务
2. **WebSocket支持**：正确配置Upgrade和Connection头，支持HTTP到WebSocket的协议升级
3. **超时设置**：设置合适的超时时间，平衡资源利用和连接稳定性
4. **健康检查**：配置后端服务健康检查，自动剔除故障节点
5. **SSL终止**：在Nginx层处理SSL加密，减轻后端服务负担

性能优化：

- 关闭代理缓冲，减少延迟
- 设置合理的worker进程数和连接数
- 优化内核参数，提高并发处理能力

```
# nginx.conf - 高性能WebSocket负载均衡配置
upstream dex_websocket {
    # 使用一致性哈希确保会话粘性
    hash $remote_addr consistent;

    server dex-websocket-1:8080 max_fails=3 fail_timeout=30s;
    server dex-websocket-2:8080 max_fails=3 fail_timeout=30s;
    server dex-websocket-3:8080 max_fails=3 fail_timeout=30s;
}

server {
    listen 80;
    listen 443 ssl http2;
    server_name api.yourdex.com;

    # SSL配置
    ssl_certificate /etc/nginx/ssl/cert.pem;
    ssl_certificate_key /etc/nginx/ssl/key.pem;

    # WebSocket代理
    location /ws {
        proxy_pass http://dex_websocket;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
```



```

    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;

    # 超时设置
    proxy_connect_timeout 60s;
    proxy_send_timeout 60s;
    proxy_read_timeout 3600s;

    # 缓冲设置
    proxy_buffering off;
    proxy_request_buffering off;
}

# API代理
location /api {
    proxy_pass http://dex_websocket;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}

# 健康检查
location /health {
    proxy_pass http://dex_websocket;
    access_log off;
}
}

```

3. 监控配置

Prometheus监控

监控体系说明：

完善的监控体系是保证系统稳定运行的关键。通过Prometheus收集指标，Grafana进行可视化展示，AlertManager处理告警通知，形成完整的监控闭环。

监控指标类别：

1. 系统指标：CPU使用率、内存使用率、磁盘I/O、网络流量
2. 应用指标：WebSocket连接数、消息吞吐量、处理延迟、错误率
3. 业务指标：K线更新频率、大额交易检测数量、用户活跃度
4. 基础设施指标：Redis性能、Nginx状态、Docker容器状态

告警策略：

- 连接数超过阈值时告警
- 消息处理延迟过高时告警
- 错误率超过1%时告警
- 服务不可用时立即告警

数据保留:

- 高频指标保留7天
- 日级聚合数据保留30天
- 月级聚合数据保留1年

```
# prometheus.yml - 监控配置
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'dex-websocket'
    static_configs:
      - targets: ['dex-websocket-1:8080', 'dex-websocket-2:8080', 'dex-websocket-3:8080']
    scrape_interval: 5s
    metrics_path: /metrics

  - job_name: 'redis'
    static_configs:
      - targets: ['redis:6379']

  - job_name: 'nginx'
    static_configs:
      - targets: ['nginx:9113']
```

Grafana仪表板

```
{
  "dashboard": {
    "title": "DEX WebSocket监控",
    "panels": [
      {
        "title": "活跃连接数",
        "type": "stat",
        "targets": [
          {
            "expr": "sum(websocket_connections_active)",
            "legendFormat": "总连接数"
          }
        ]
      },
      {
        "title": "消息处理速度",
        "type": "graph",
        "targets": [
          {
            "expr": "sum(rate(websocket_messages_total[1m])) by (type)",
            "legendFormat": "{{type}}"
          }
        ]
      }
    ]
  },
}
```

```
{
  "title": "延迟分布",
  "type": "heatmap",
  "targets": [
    {
      "expr": "histogram_quantile(0.95,
sum(rate(websocket_message_latency_seconds_bucket[5m])) by (le))",
      "legendFormat": "95th percentile"
    }
  ]
}
```

性能测试与优化

压力测试脚本

测试目标：

验证系统在高并发场景下的稳定性和性能表现，确保能够稳定支持1万+并发WebSocket连接，并保持低延迟的消息传输。

测试方案：

1. **连接测试**：逐步建立大量WebSocket连接，测试系统的连接承载能力
2. **消息测试**：在高并发连接下发送大量消息，测试消息处理能力
3. **稳定性测试**：长时间运行测试，验证系统的稳定性
4. **故障恢复测试**：模拟网络断开和重连，测试自动重连功能

关键指标：

- 最大并发连接数
- 消息处理延迟（P50、P95、P99）
- 消息吞吐量（消息/秒）
- 内存使用情况
- CPU使用率
- 连接建立成功率
- 重连成功率

测试环境要求：

- 测试机器需要足够的文件描述符限制
- 网络带宽充足
- 系统资源充足（CPU、内存）

```
package main

import (
    "flag"
```

```

    "fmt"
    "log"
    "sync"
    "sync/atomic"
    "time"

    "github.com/gorilla/websocket"
)

var (
    connections = flag.Int("connections", 1000, "并发连接数")
    duration    = flag.Duration("duration", 60*time.Second, "测试持续时间")
    url         = flag.String("url", "ws://localhost:8080/ws", "WebSocket URL")
)

func main() {
    flag.Parse()

    var (
        connected    int64
        disconnected  int64
        messages     int64
        errors       int64
    )

    var wg sync.WaitGroup

    // 启动连接
    for i := 0; i < *connections; i++ {
        wg.Add(1)
        go func(id int) {
            defer wg.Done()

            conn, _, err := websocket.DefaultDialer.Dial(*url, nil)
            if err != nil {
                atomic.AddInt64(&errors, 1)
                return
            }
            defer conn.Close()

            atomic.AddInt64(&connected, 1)

            // 订阅频道
            conn.WriteJSON(map[string]interface{}{
                "type": "subscribe",
                "channels": []string{"kline_ETHUSDT_1m", "price_alerts"},
            })

            // 接收消息
            done := make(chan struct{})
            go func() {
                defer close(done)

```

```

        for {
            _, _, err := conn.ReadMessage()
            if err != nil {
                return
            }
            atomic.AddInt64(&messages, 1)
        }
    }()

    // 等待测试结束
    select {
    case <-done:
    case <-time.After(*duration):
    }

    atomic.AddInt64(&disconnected, 1)
}(i)

// 控制连接建立速度
if i%100 == 0 {
    time.Sleep(100 * time.Millisecond)
}
}

// 统计协程
go func() {
    ticker := time.NewTicker(time.Second)
    defer ticker.Stop()

    var lastMessages int64

    for range ticker.C {
        currentMessages := atomic.LoadInt64(&messages)
        mps := currentMessages - lastMessages
        lastMessages = currentMessages

        fmt.Printf("连接: %d, 断开: %d, 消息/秒: %d, 总消息: %d, 错误: %d\n",
            atomic.LoadInt64(&connected),
            atomic.LoadInt64(&disconnected),
            mps,
            currentMessages,
            atomic.LoadInt64(&errors))
    }
}()

wg.Wait()

fmt.Printf("\n测试完成:\n")
fmt.Printf("总连接: %d\n", atomic.LoadInt64(&connected))
fmt.Printf("总消息: %d\n", atomic.LoadInt64(&messages))
fmt.Printf("错误数: %d\n", atomic.LoadInt64(&errors))

```

```
    fmt.Printf("平均消息/秒: %.2f\n",  
float64(atomic.LoadInt64(&messages))/duration.Seconds())  
}
```